Decrease by a Constant Factor

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Decrease-by-Constant-Factor Algorithms

In this variation of decrease-and-conquer, instance size is reduced by the same factor (typically, 2)

Examples:

- Binary search
- Method of bisection
- Exponentiation by squaring
- Multiplication à la russe (Russian peasant method)
- Fake-coin puzzle
- Josephus problem

Binary Search

Very efficient algorithm for searching in sorted array:

K

vs

A[0] . . . A[m] . . . A[n-1]

If K = A[m], stop (successful search); otherwise, continue searching by the same method in A[0..m-1] if K < A[m] and in A[m+1..n-1] if K > A[m]

```
l ← 0; r ← n-1
while l ≤ r do
   m ← ⌊(l+r)/2⌋
   if K = A[m] return m
   else if K < A[m] r ← m-1
   else l ← m+1
return -1
```

Analysis of Binary Search

- Time efficiency
  - worst-case recurrence: $C_w(n) = 1 + C_w(\lfloor n/2 \rfloor)$, $C_w(1) = 1$
    solution: $C_w(n) = \lceil \log_2(n+1) \rceil$
  - This is VERY fast: e.g., $C_w(10^6) = 20$

- Optimal for searching a sorted array

- Limitations: must be a sorted array (not linked list)

- Has a continuous counterpart called the bisection method for solving equations in one unknown $f(x) = 0$

Exponentiation by Squaring

The problem: Compute a^n where n is a non-negative integer

The problem can be solved by applying recursively the formulas:

For even values of n: $a^n = (a^{n/2})^2$ if $n > 0$ and $a^0 = 1$

For odd values of n: $a^n = (a^{(n-1)/2})^2 \cdot a$

Recurrence: $M(n) = M(\lfloor n/2 \rfloor) + f(n)$, where $f(n) = 1$ or 2,
$M(0) = 0$
Master Theorem: $M(n) \in \Theta(\log n) = \Theta(b)$ where $b = \lceil \log_2(n+1) \rceil$

Russian Peasant Multiplication

The problem: Compute the product of two positive integers

Can be solved by a decrease-by-half algorithm based on the following formulas.

For even values of n:

$$n * m = n/2 * 2m$$

For odd values of n:

$$n * m = (n-1)/2 * 2m + m \quad \text{if } n > 1 \qquad \text{and} \qquad m \quad \text{if } n = 1$$

Example of Russian Peasant Multiplication

```
Compute   20 * 26
           n   m
          20   26
          10   52
           5   104   104
           2   208   +
           1   416   416
                     520
```

Note: Method reduces to adding m's values corresponding to odd n's.

Fake-Coin Puzzle (simpler version)

There are n identically looking coins one of which is fake. There is a balance scale but there are no weights; the scale can tell whether two sets of coins weigh the same and, if not, which of the two sets is heavier (but not by how much). Design an efficient algorithm for detecting the fake coin. Assume that
the fake coin is known to be lighter than the genuine ones.

Decrease by factor 2 algorithm

Decrease by factor 3 algorithm

Variable-size Decrease

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Variable-Size-Decrease Algorithms

In the variable-size-decrease variation of decrease-and-conquer, instance size reduction varies from one iteration to another

Examples:

- Euclid's algorithm for greatest common divisor
- Partition-based algorithm for the selection problem
- Interpolation search
- Algorithms on binary search trees
- Nim and Nim-like games

Euclid’s Algorithm

Euclid’s algorithm is based on repeated application of equality

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

Ex.: $\gcd(80,44) = \gcd(44,36) = \gcd(36, 12) = \gcd(12,0) = 12$

One can prove that the size, measured by the second number, decreases at least by half after two consecutive iterations.
Hence, $T(n) \in O(\log n)$

Selection Problem

Find the k-th smallest element in a list of n numbers

- $k = 1$ or $k = n$

- <u>median</u>: $k = \lceil n/2 \rceil$
  Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = ?

The median is used in statistics as a measure of an average value of a sample. In fact, it is a better (more robust) indicator than the mean, which is used for the same purpose.

Algorithms for the Selection Problem

The sorting-based algorithm: Sort and return the k-th element
Efficiency (if sorted by mergesort): O(n log n)

A faster algorithm is based on the array partitioning:

Assuming that the array is indexed from 0 to n-1 and s is a split position obtained by the array partitioning:
If s = k-1, the problem is solved;
if s > k-1, look for the k-th smallest element in the left part;
if s < k-1, look for the (k-s)-th smallest element in the right part.

Note: The algorithm can simply continue until s = k-1.

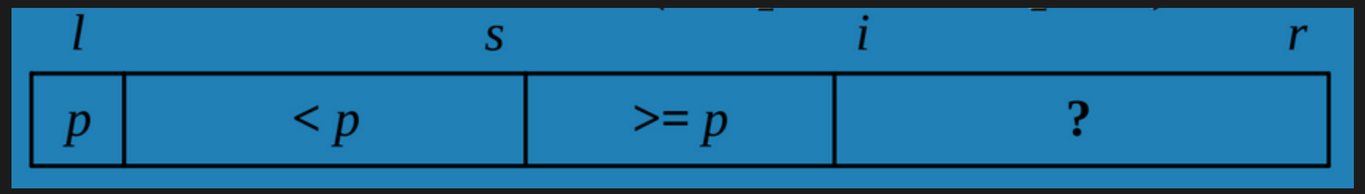
Two Partitioning Algorithms

There are two principal ways to partition an array:

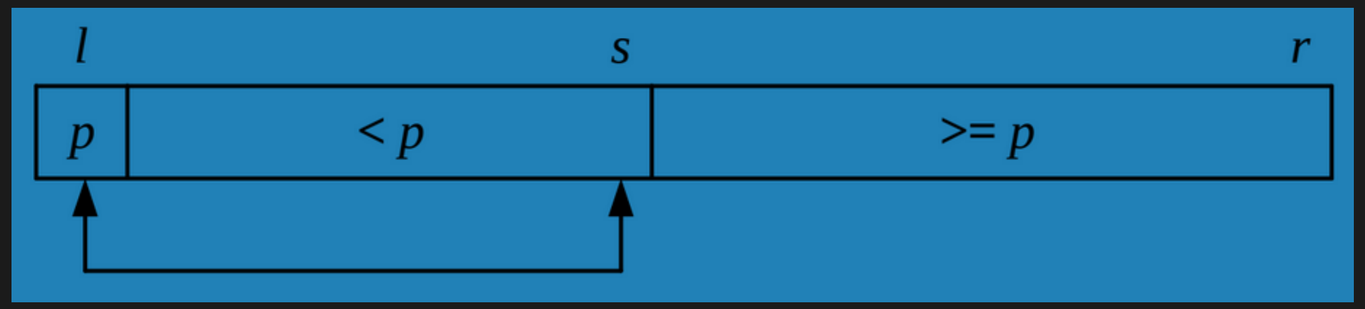
- One-directional scan (Lomuto’s partitioning algorithm)
- Two-directional scan (Hoare’s partitioning algorithm)

Lomuto’s Partitioning Algorithm

Scans the array left to right maintaining the array’s partition into three contiguous sections: < p, ≥ p, and unknown, where p is the value of the first element (the partition’s pivot).



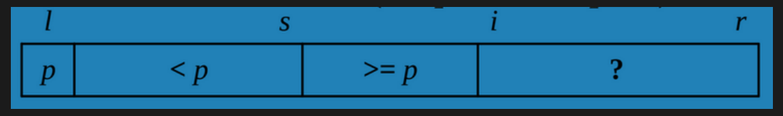
On each iteration the unknown section is decreased by one element until it’s empty and a partition is achieved by exchanging the pivot with the element in the split position s.

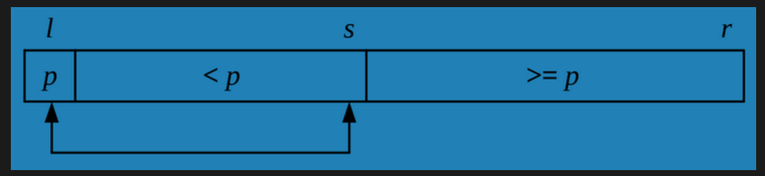


Lomuto’s Partitioning Algorithm

```
ALGORITHM LomutoPartition(A[l..r])
//Partitions subarray by Lomuto’s algorithm using first element as pivot
//Input: A subarray A[l..r] of array A[0..n - 1], defined by its left and
//       right indices l and r (l ≤ r)
//Output: Partition of A[l..r] and the new position of the pivot

  p ← A[l]
  s ← l
  for i ← l + 1 to r do
    if A[i] < p
      s ← s + 1
      swap(A[s], A[i])
  swap(A[l], A[s])
  return s
```





Tracing Lomuto’s Partioning Algorithm

s	i							
4	1	10	8	7	12	9	2	15
	s	i						
4	1	10	8	7	12	9	2	15
	s						i	
4	1	10	8	7	12	9	2	15
		s						i
4	1	2	8	7	12	9	10	15
		s						
4	1	2	8	7	12	9	10	15
2	1	4	8	7	12	9	10	15

Tracing Quickselect (Partition-based Algorithm)

Find the median of 4, 1, 10, 9, 7, 12, 8, 2, 15

Here: n = 9, k = ⌈9/2⌉ = 5, k -1=4

0	1	2	3	**4**	5	6	7	8
4	1	10	8	7	12	9	2	15
2	1	**4**	8	7	12	9	10	15
			8	7	12	9	10	15
			7	**8**	12	9	10	15

after 1st partitioning: s=2<k-1=4

after 2nd partitioning: s=4=k-1

The median is A[4]= 8

Efficiency of Quickselect

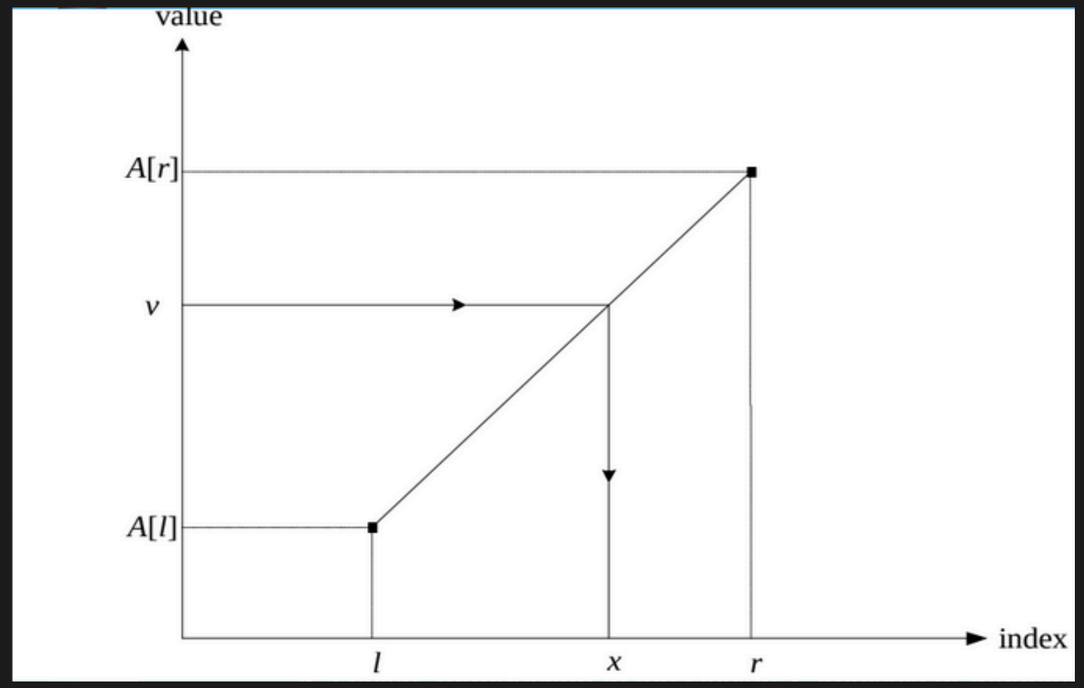
Average case (average split in the middle):
$C(n) = C(n/2)+(n+1)$ $C(n) \in \Theta(n)$

Worst case (degenerate split): $C(n) \in \Theta(n^2)$

A more sophisticated choice of the pivot leads to a complicated algorithm with $\Theta(n)$ worst-case efficiency.

Interpolation Search

Searches a sorted array similar to binary search but estimates location of the search key in A[l…r] by using its value v. Specifically, the values of the array's elements are assumed to grow linearly from A[l] to A[r] and the location of v is estimated as the x-coordinate of the point on the straight line through (l, A[l]) and (r, A[r]) whose y-coordinate is v:



$$x = l + \lfloor (v - A[\,l\,])(r - l) \,/\, (A[\,r\,] - A[\,l\,]) \rfloor$$

Analysis of Interpolation Search

- Efficiency

  average case: $C(n) < \log_2 \log_2 n + 1$

  worst case: $C(n) = n$

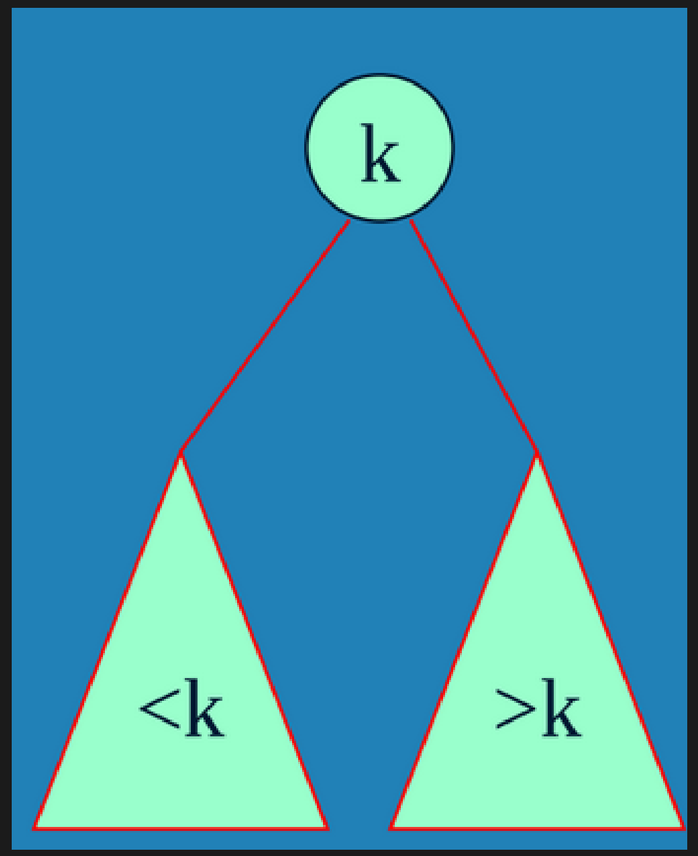
- Preferable to binary search only for VERY large arrays and/or expensive comparisons

- Has a counterpart, the method of false position (regula falsi), for solving equations in one unknown

Binary Search Tree Algorithms

Several algorithms on BST requires recursive processing of just one of its subtrees, e.g.,

- Searching
- Insertion of a new key
- Finding the smallest (or the largest) key



Searching in Binary Search Tree

ALGORITHM BTS(x, v)

//Searches for node with key equal to v in BST rooted at node x

if x = NIL **return** -1

else if v = K(x) **return** x

else if v < K(x) **return** *BTS(left(x), v)*

else return *BTS(right(x), v)*

Efficiency

worst case: $C(n) = n$

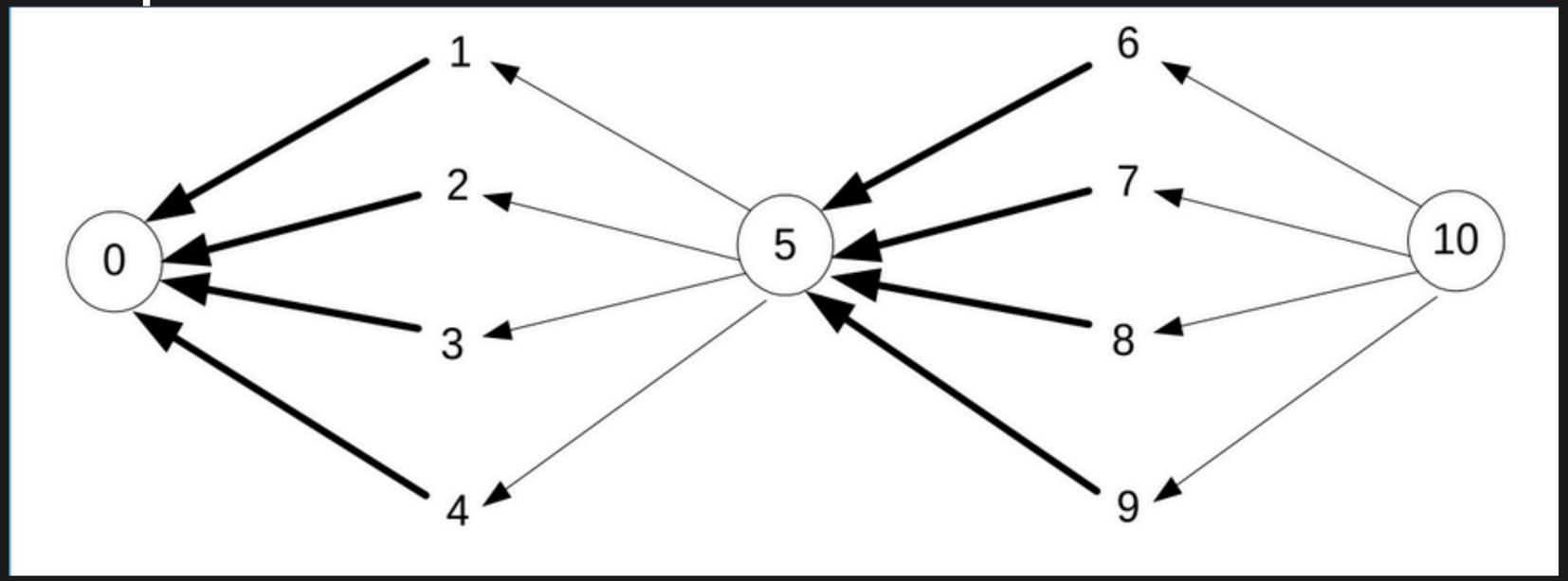
average case: $C(n) \approx 2\ln n \approx 1.39\log_2 n$

One-Pile Nim

There is a pile of n chips. Two players take turns by removing from the pile at least 1 and at most m chips. (The number of chips taken can vary from move to move.) The winner is the player that takes the last chip. Who wins the game – the player moving first or second, if both players make the best moves
possible?

It's a good idea to analyze this and similar games "backwards", i.e., starting with n = 0, 1, 2, ...

Partial Graph of One-Pile Nim with m = 4



Vertex numbers indicate n, the number of chips in the pile. The losing position for the player to move are circled. Only winning moves from a winning position are shown (in bold).

<u>Generalization:</u> The player moving first wins iff n is not a multiple of 5 (more generally, m+1); the winning move is to take n mod 5 (n mod (m+1)) chips on every move.

Q & A

FOR QUESTIONS OR COMMENTS

EMAIL ADDRESS

ipineda@yachaytech.edu.ec